МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Казанский национальный исследовательский технический университет им. А.Н. Туполева-КАИ» (КНИТУ-КАИ)
Институт Компьютерных технологий и защиты информации

Кафедра Систем информационной безопасности

**ОТЧЁТ**

по лабораторной работе №1
по дисциплине
«Аппаратные средства телекоммуникационных систем»
на тему:

# «ОСНОВЫ OPENMP: ПАРАЛЛЕЛЬНЫЕ ОБЛАСТИ, РАСПРЕДЕЛЕНИЕ РАБОТ, СИНХРОНИЗАЦИЯ ПОТОКОВ»

Выполнил:
Студент гр. 4404
Устеников К. А.

Казань, 2026

Цель работы:

Освоение базовых приёмов параллельного программирования для архитектур с разделяемой памятью: организация параллельных областей и нумерация потоков, распределение итераций цикла между потоками разными способами (аналог schedule в OpenMP), а также применение примитивов синхронизации. Выполнены задачи 1–3 из пособия «Параллельное программирование на OpenMP».

Поскольку работа реализуется на языке C#, в котором директивы компилятора OpenMP отсутствуют, те же самые конструкции воспроизводятся штатными средствами языка. Соответствие приведено в таблице:

| Конструкция OpenMP | Средство C#, используемое в работе |
|---|---|
| команда потоков (omp parallel, модель fork-join) | массив Thread[] и вызов Start(); размер команды задаётся аргументом командной строки (аналог OMP_NUM_THREADS) |
| неявный барьер на выходе из параллельной области | цикл Join() над всеми потоками |
| omp_get_thread_num() | номер id, передаваемый в поток параметром |
| критическая секция (omp critical) | блок lock (obj) { … } |
| атомарные операции (atomic) | методы Interlocked.* и Volatile.* |
| фиксация порядка (ordered) | условная переменная Monitor.Wait/PulseAll, каналы System.Threading.Channels, семафоры SemaphoreSlim, события ManualResetEventSlim, активное ожидание |
| переменная окружения OMP_SCHEDULE | Environment.GetEnvironmentVariable("LAB2_SCHEDULE") |

Задание:

Задача 1. Реализовать программу, в которой каждый поток выводит собственный идентификатор, общее число потоков и строку «Hello World». Выполнить запуск с 8 потоками. Определить, всегда ли вывод совпадает от запуска к запуску, и объяснить причину.

Задача 2. Реализовать программу с массивом a из 16000 элементов, где a[i] = i. Построить результирующий массив b, в котором для внутренних позиций вычисляется среднее трёх соседних значений: b[i] = (a[i-1] + a[i] + a[i+1]) / 3.0. Выполнить запуск с 8 потоками при разных типах распределения работ.

Задача 3. Изменить программу задачи 1 так, чтобы идентификаторы выводились в обратном порядке (от N-1 до 0). Известно не менее пяти способов решения — реализовать и показать максимально возможное их число.

Общие требования. Количество потоков и размер задачи подаются программе через командную строку. Применение библиотечных средств, решающих задачу целиком, запрещено. Каждый запуск, упомянутый в условии (разное число потоков, разные типы расписания, разные версии программы), сопровождается скриншотом, на котором видна команда запуска и полный вывод без обрезки и правок.

Ход работы:

Задача 1. Параллельная область и идентификация потоков.

На входе программа принимает из аргументов командной строки количество потоков N и создаёт ровно N экземпляров класса Thread — это прямой аналог команды потоков OpenMP (в C# нет вызова вроде GOMAXPROCS, размер команды задаётся явно). Каждый поток печатает свой номер, общее число потоков и текст Hello World. Главный поток затем в цикле вызывает Join() для каждого рабочего потока, воспроизводя неявный барьер, который OpenMP ставит на выходе из параллельной области. Конструкция lock окружает лишь операцию записи в консоль и нужна исключительно для того, чтобы строки разных потоков не слились посимвольно; очерёдность самих строк она не задаёт.

Фрагмент кода (lab1/task1/Program.cs):

```
for (int i = 0; i < numThreads; i++)
{
    int id = i;                         // копия индекса для замыкания
    threads[i] = new Thread(() =>
    {
        lock (printLock)                // защита операции печати
        {
            Console.WriteLine($"Thread{id} of {numThreads}: Hello World");
```

```
        }
    });
    threads[i].Start();              // fork
}
```

foreach (var t in threads) t.Join();  // неявный барьер (join)

Команды запуска (три повтора одной команды):

```
cd lab1/task1
dotnet run -- 8
dotnet run -- 8
dotnet run -- 8
```

Вывод не всегда идентичен. Потоки стартуют практически одновременно и исполняются параллельно, поэтому последовательность, в которой каждый из них доходит до оператора печати, выбирает планировщик операционной системы, а не программа. Во всех запусках печатается один и тот же набор строк (по одной от каждого из 8 потоков), но их порядок меняется. lock обеспечивает лишь атомарность записи строки, не внося упорядоченности между потоками; синхронизация, фиксирующая порядок, в задаче 1 сознательно отсутствует. Для подтверждения сделано три запуска с 8 потоками.

На рисунке 1 представлен первый запуск: видна команда и полный вывод — все 8 потоков напечатали свой идентификатор, общее число потоков и Hello World.

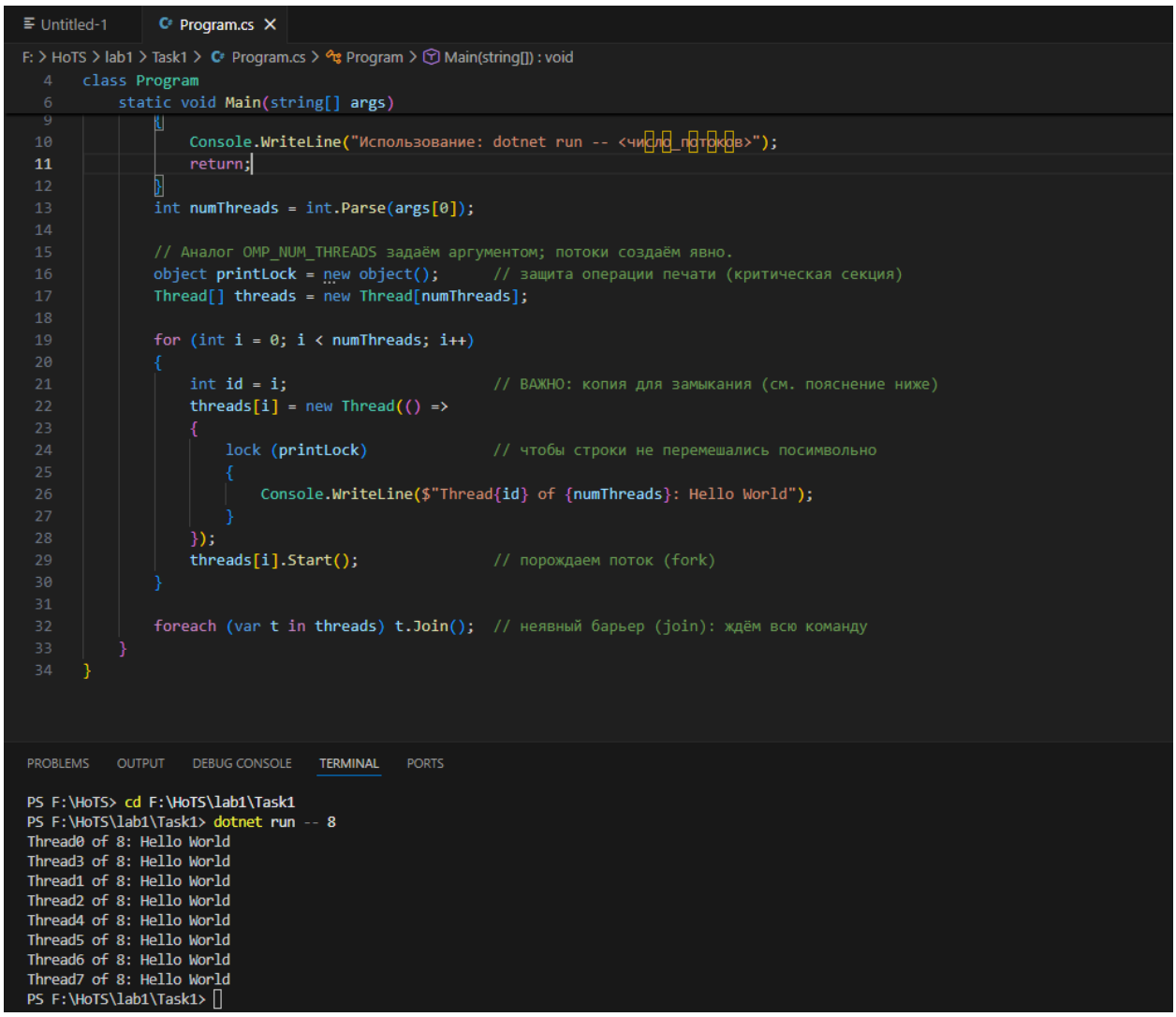



*Рисунок 1. Запуск с 8 потоками, запуск 1*

На рисунке 2 второй запуск той же команды. Порядок строк, как правило, отличается от рисунка 1, поскольку очерёдность выполнения потоков программой не определена. Совпадение порядка в отдельных запусках не означает детерминированности: программа его не фиксирует.

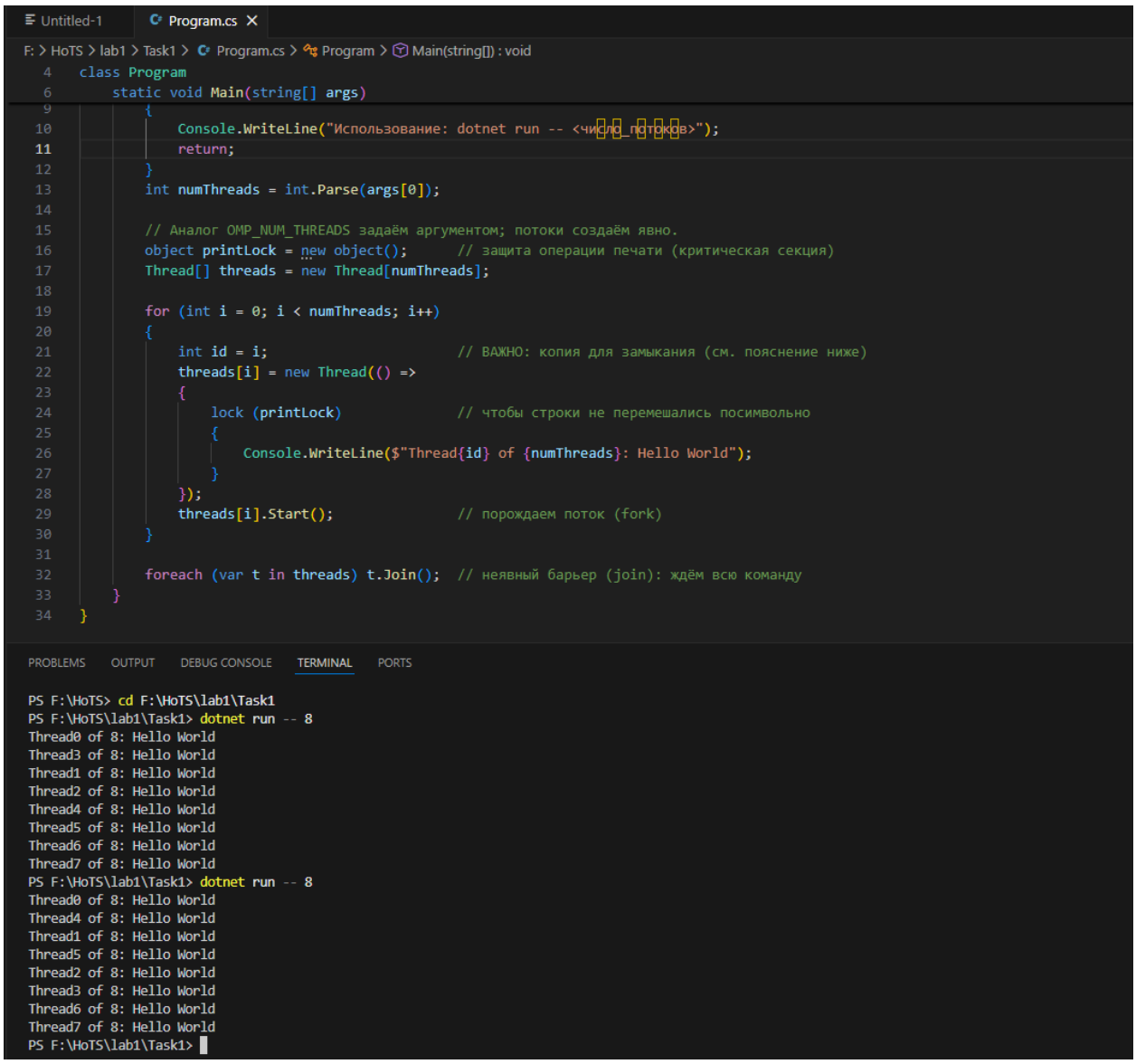



*Рисунок 2. Запуск с 8 потоками, запуск 2*

На рисунке 3 третий запуск. Сопоставление рисунков 1–3 показывает неизменность набора строк при изменении их последовательности, что подтверждает ответ на вопрос.

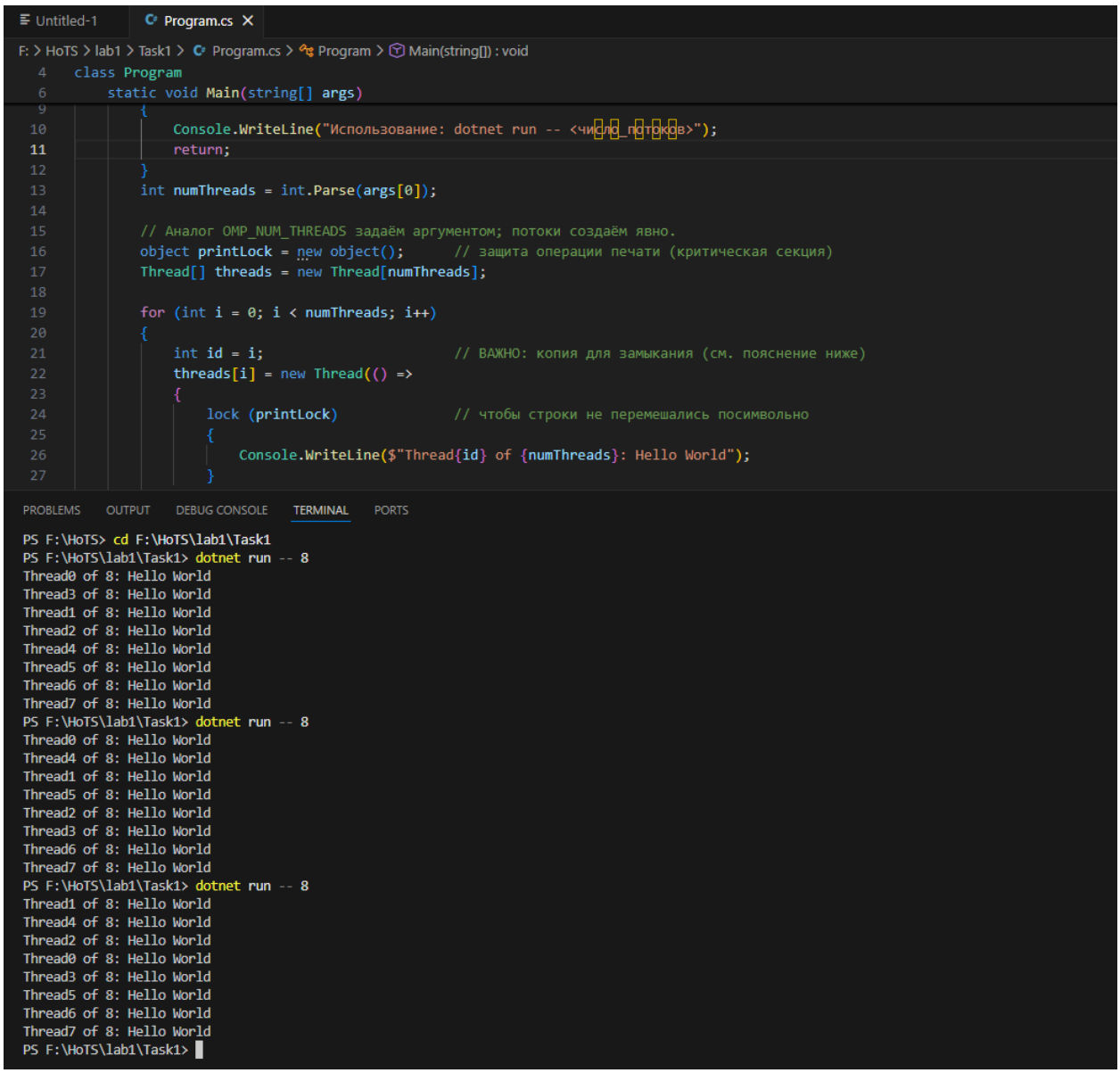



*Рисунок 3. Запуск с 8 потоками, запуск 3*

Задача 1. Параллельная область и идентификация потоков.

Программа формирует массив a длины N (N = 16000 приходит из командной строки) с a[i] = i и вычисляет массив b по формуле b[i] = (a[i-1] + a[i] + a[i+1]) / 3.0 для внутренних индексов i = 1 … N-2; краевые позиции b[0] и b[N-1] условием не определены, поэтому в них копируются значения из a. Обработка внутренних элементов распределяется между 8 потоками в соответствии с выбранным типом расписания, повторяющим логику schedule из OpenMP:

- static без блока — интервал итераций делится на непрерывные равные участки, каждый поток получает свой отрезок;

- static с блоком chunk — итерации режутся на блоки фиксированного размера и раздаются потокам циклически (блок 0 → поток 0, блок 1 → поток 1, и т. д. по кругу);
- dynamic — блоки выдаются из общей очереди, защищённой lock (аналог критической секции); освободившийся поток захватывает следующий блок;
- guided — как dynamic, но размер выдаваемого блока постепенно уменьшается по мере съедания массива, не опускаясь ниже chunk;
- runtime — тип расписания и размер блока читаются из переменной окружения LAB2_SCHEDULE (аналог OMP_SCHEDULE).

Фрагмент кода (lab1/task2/Program.cs, выдача блока в dynamic/guided):

```
lock (queueLock)
{
   if (next <= endIdx)
   {
      s = next;
      e = Math.Min(s + chunk - 1, endIdx);
      next = e + 1;
      got = true;
   }
}
if (!got) break;
Process(s, e);   // b[i] = (a[i-1] + a[i] + a[i+1]) / 3.0
```

Конфликтов доступа (гонок) здесь нет: каждый поток пишет лишь в принадлежащие ему индексы массива b, тогда как массив a неизменен и доступен только на чтение. Следовательно, итоговый массив и его контрольная сумма не зависят от выбранного расписания (при a[i] = i получается b[i] = i, а сумма равна N·(N-1)/2 = 127992000; программа печатает OK: True). Различаются только распределение итераций и блоков по потокам и время выполнения.

Команды запуска (5 запусков с 8 потоками):

```
cd lab1/task2
```

dotnet run -- 8 16000 static

dotnet run -- 8 16000 static 1000

dotnet run -- 8 16000 dynamic 100

dotnet run -- 8 16000 guided 100

$env:LAB2_SCHEDULE = "dynamic,100"; dotnet run -- 8 16000 runtime

На рисунке 4 расписание static без блока: 15998 внутренних итераций разбиты на непрерывные участки (6 потоков по 2000 итераций, 2 потока по 1999). Контрольная сумма совпала с ожидаемой (OK: True).

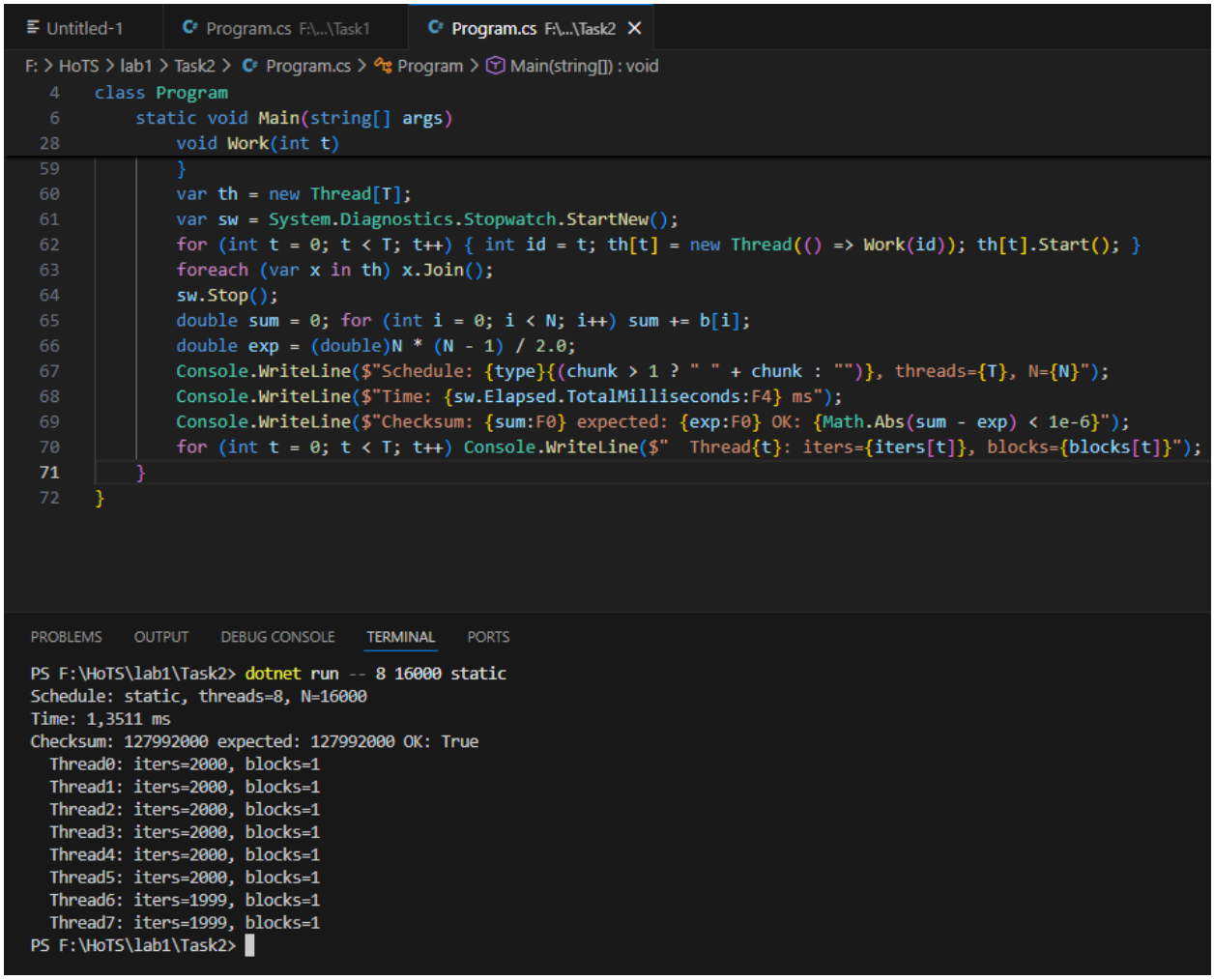



*Рисунок 4. 8 потоков, расписание static (без блока)*

На рисунке 5 static с блоком 1000: итерации раздаются циклически блоками по 1000, каждый поток получает несколько блоков. Результат идентичен рисунку 4.

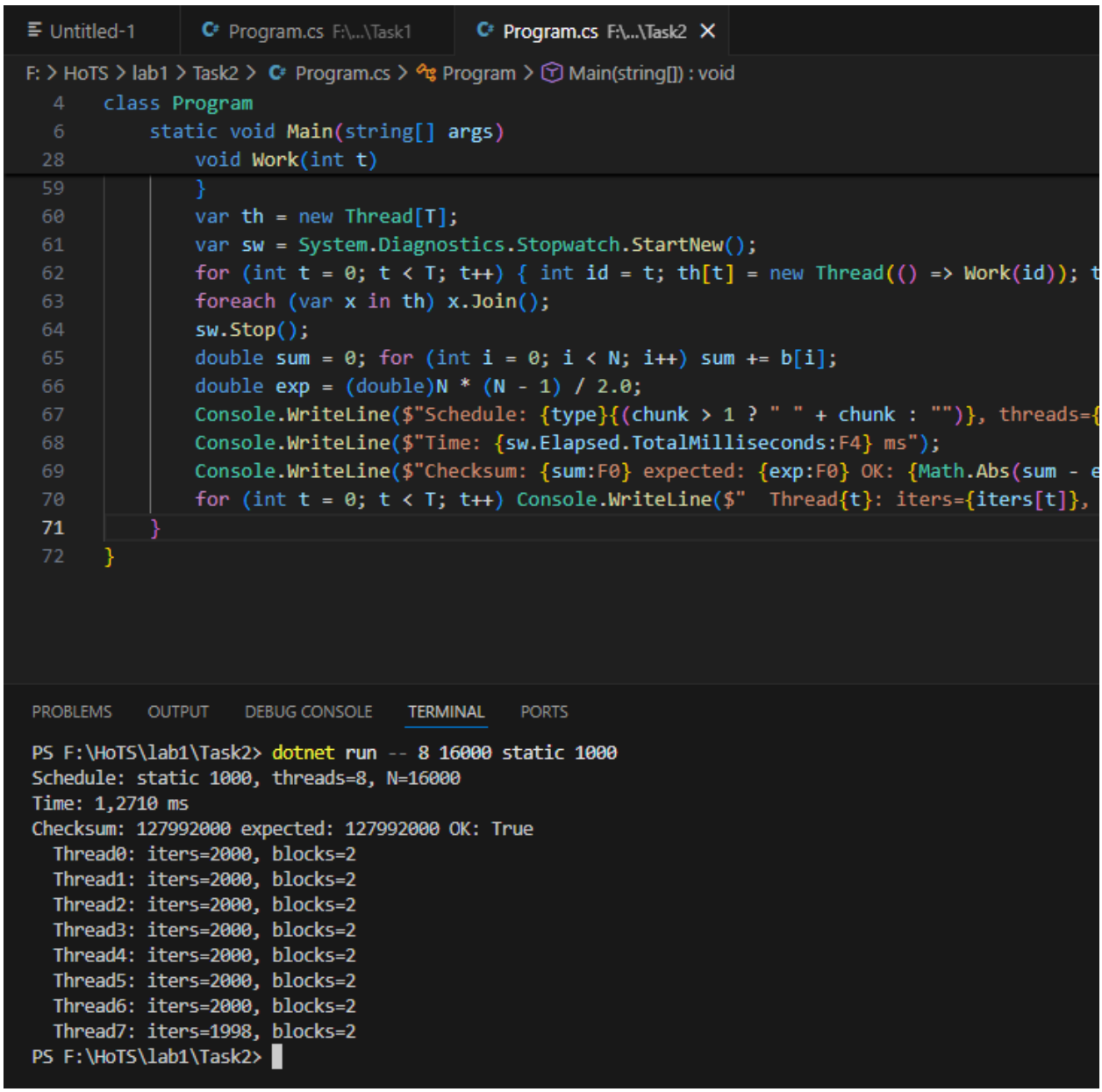



*Рисунок 5. 8 потоков, расписание static с блоком 1000*

На рисунке 6 dynamic с блоком 100: потоки берут блоки из общей очереди по мере освобождения, поэтому число блоков у разных потоков различается. Все 15998 итераций обработаны, контрольная сумма совпала.

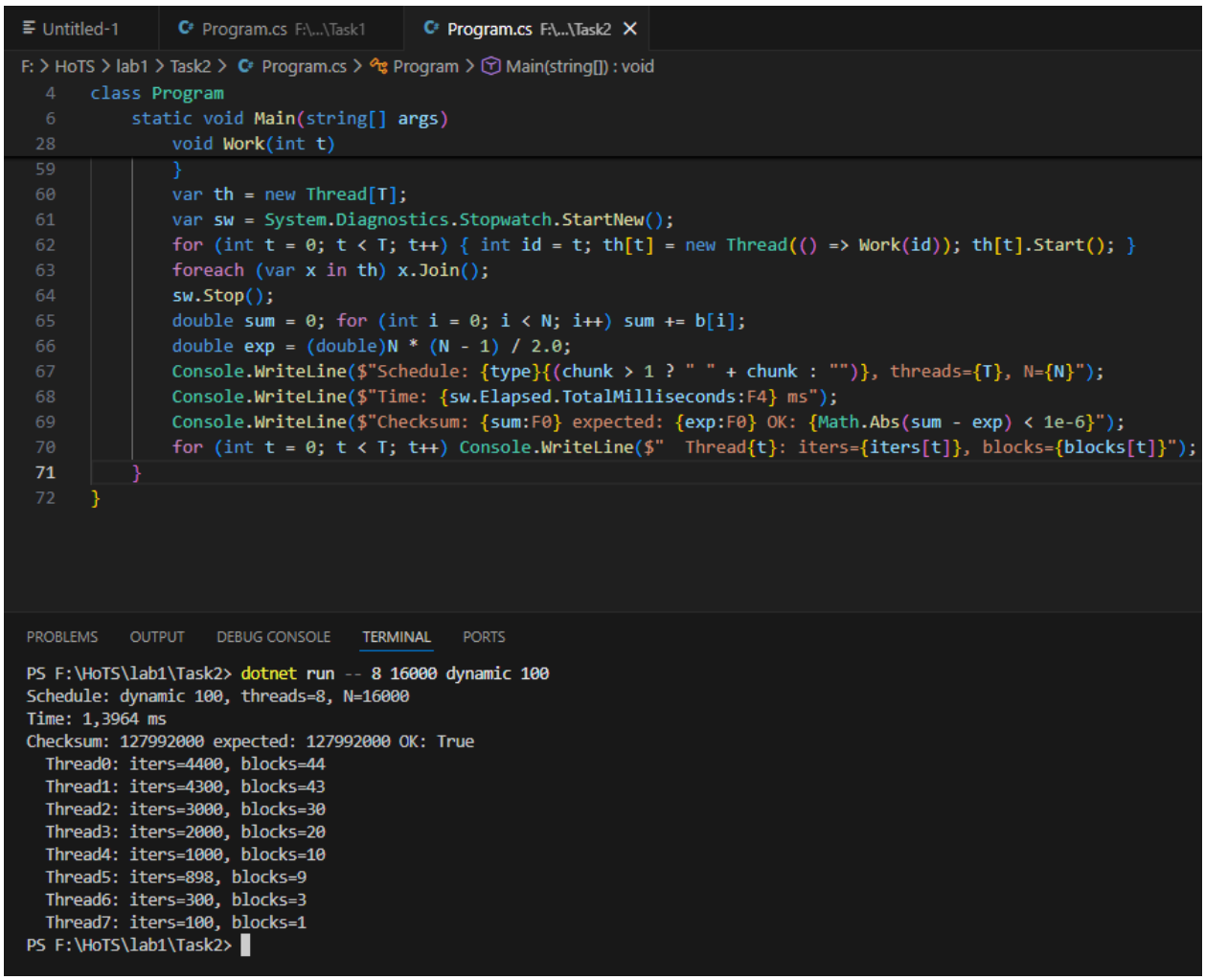



*Рисунок 6. 8 потоков, расписание dynamic с блоком 100*

На рисунке 7 guided с минимальным блоком 100: размер выдаваемых блоков уменьшается по ходу обработки, что балансирует загрузку при меньших накладных расходах. Результат совпадает с предыдущими запусками.

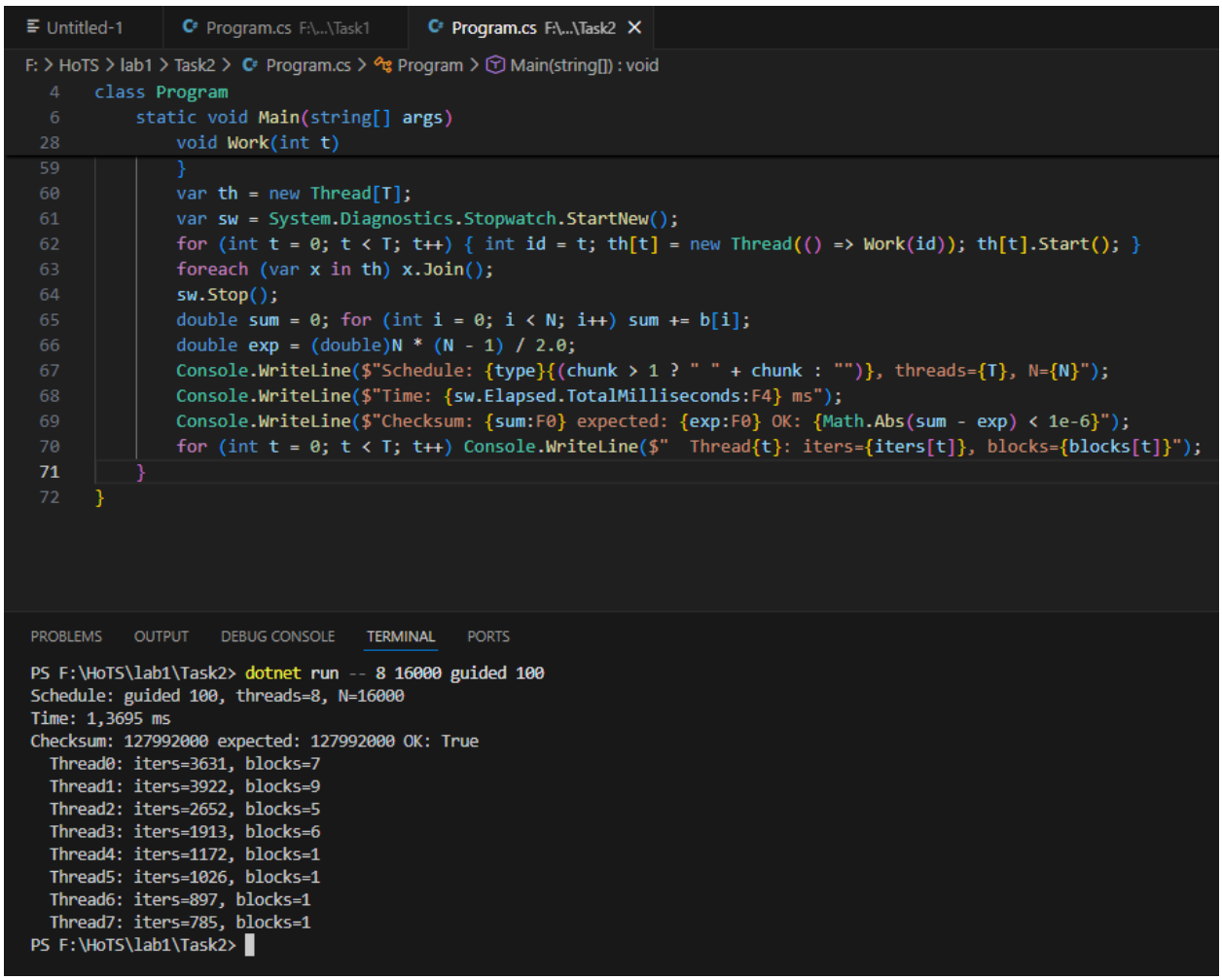



*Рисунок 7. 8 потоков, расписание guided с минимальным блоком 100*

На рисунке 8 runtime: конкретный тип распределения (dynamic,100) взят из переменной окружения LAB2_SCHEDULE, что соответствует поведению OMP_SCHEDULE в OpenMP.

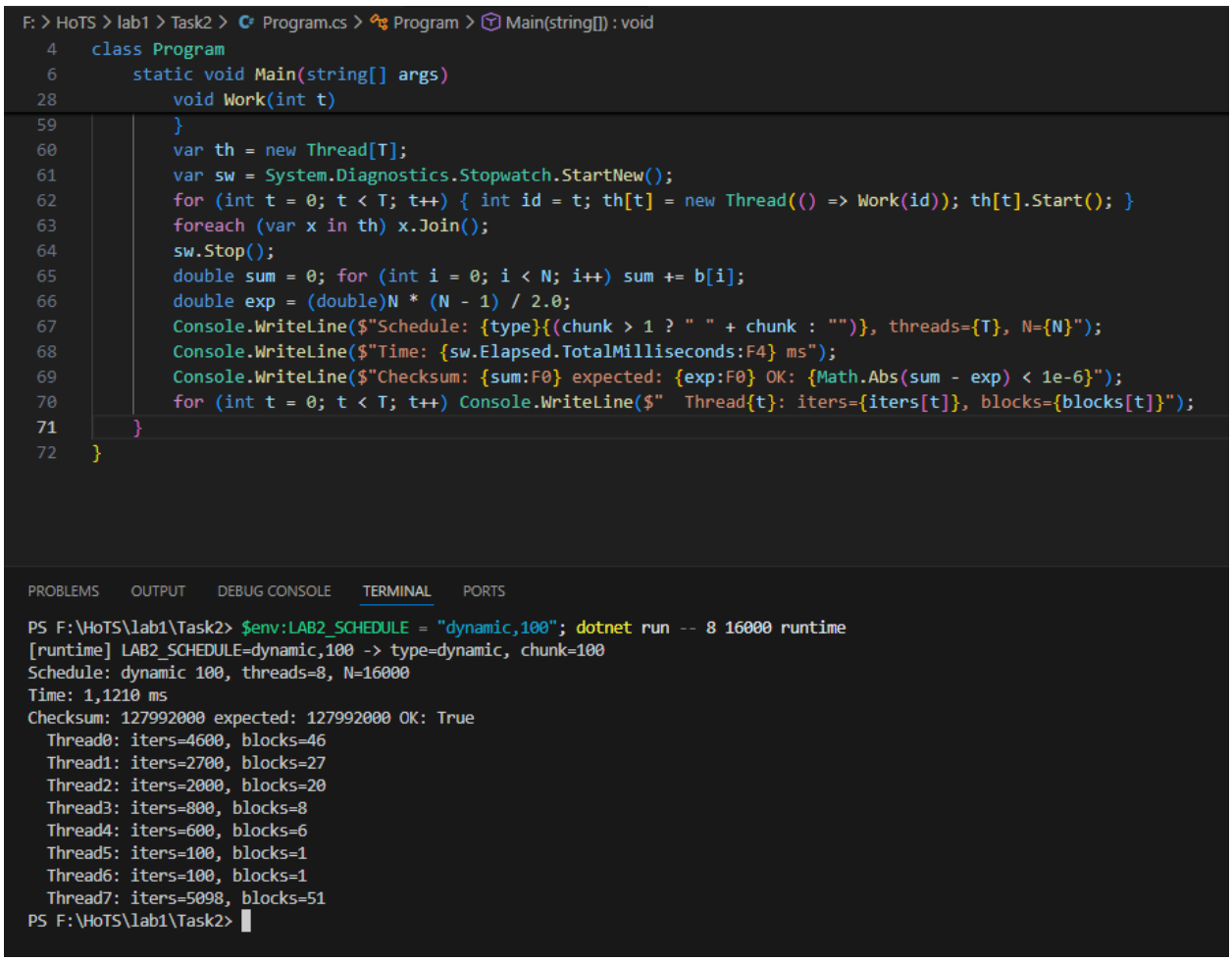



*Рисунок 8. 8 потоков, расписание runtime (dynamic,100 из окружения)*

Задача 3. Вывод идентификаторов потоков в обратном порядке.

Чтобы зафиксировать очерёдность печати, необходимы средства синхронизации. Реализовано пять способов; нужный выбирается аргументом командной строки:

- активное ожидание (spin) на атомарном счётчике (Volatile.Read / Interlocked.Decrement): поток крутится в цикле, пока счётчик не станет равен его номеру;
- условная переменная (Monitor.Wait / Monitor.PulseAll внутри lock): потоки засыпают и пробуждаются сигналом, когда наступает их очередь;
- цепочка каналов (System.Threading.Channels, передача эстафеты): поток i ждёт сигнал от потока i+1, печать начинается с N-1;

- координатор с массивом семафоров (SemaphoreSlim): главный поток выдаёт разрешения на печать в обратном порядке и дожидается подтверждения каждого потока;
- цепочка событий (ManualResetEventSlim, аналог цепочки WaitGroup): поток i ожидает, пока поток с большим номером напечатает и выставит событие.

Фрагмент кода (lab1/task3/Program.cs, способ 2):

```
lock (condObj)
{
    while (condCurrent != id) Monitor.Wait(condObj);
    Print(id, n);
    condCurrent--;
    Monitor.PulseAll(condObj);
}
```

Команды запуска (по одному способу на запуск):

```
cd lab1/task3
dotnet run -- 8 1000 spin
dotnet run -- 8 1000 cond
dotnet run -- 8 1000 chan_chain
dotnet run -- 8 1000 chan_array
dotnet run -- 8 1000 wg_chain
```

На рисунке 9 способ 1 (активное ожидание на атомарном счётчике). Идентификаторы печатаются строго в порядке 7, 6, 5, 4, 3, 2, 1, 0.

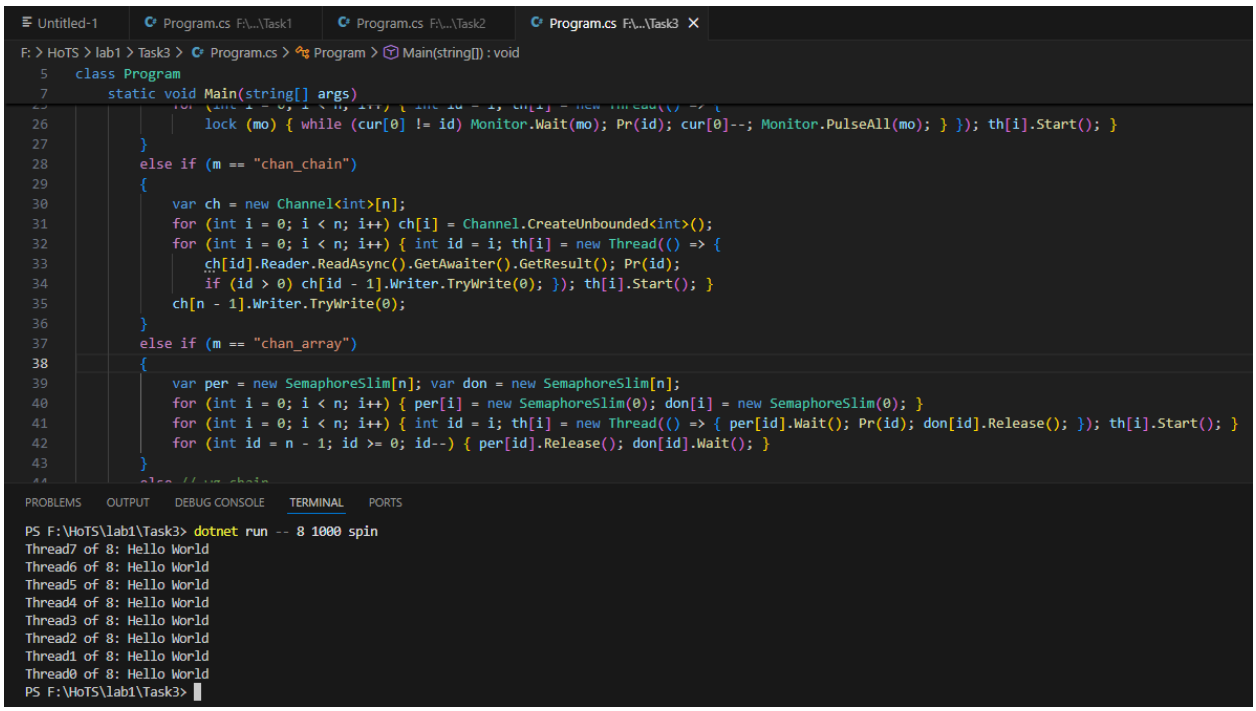



*Рисунок 9. Способ 1 (активное ожидание, атомарный счётчик)*

На рисунке 10 способ 2 (условная переменная Monitor.Wait/PulseAll). Потоки ожидают своей очереди без нагрузки на процессор; порядок вывода строго обратный.

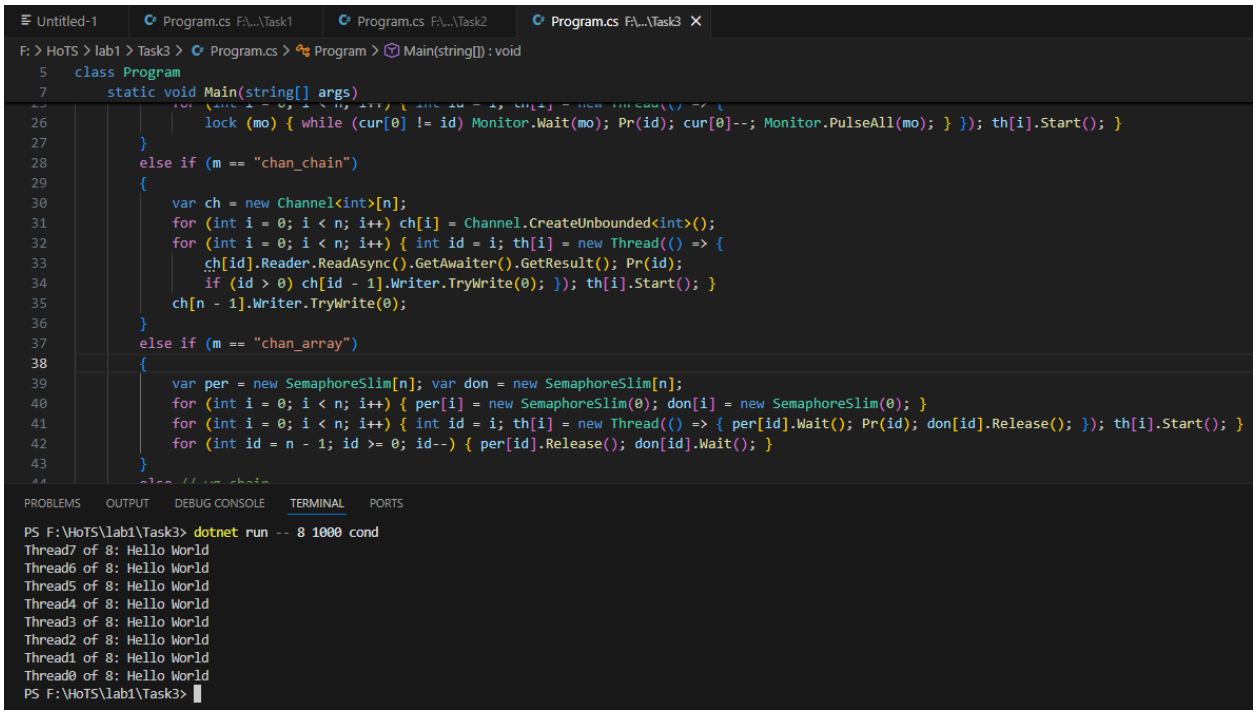



*Рисунок 10. Способ 2 (условная переменная)*

На рисунке 11 способ 3 (цепочка каналов). Право печати передаётся от потока 7 к потоку 0; порядок строго обратный.

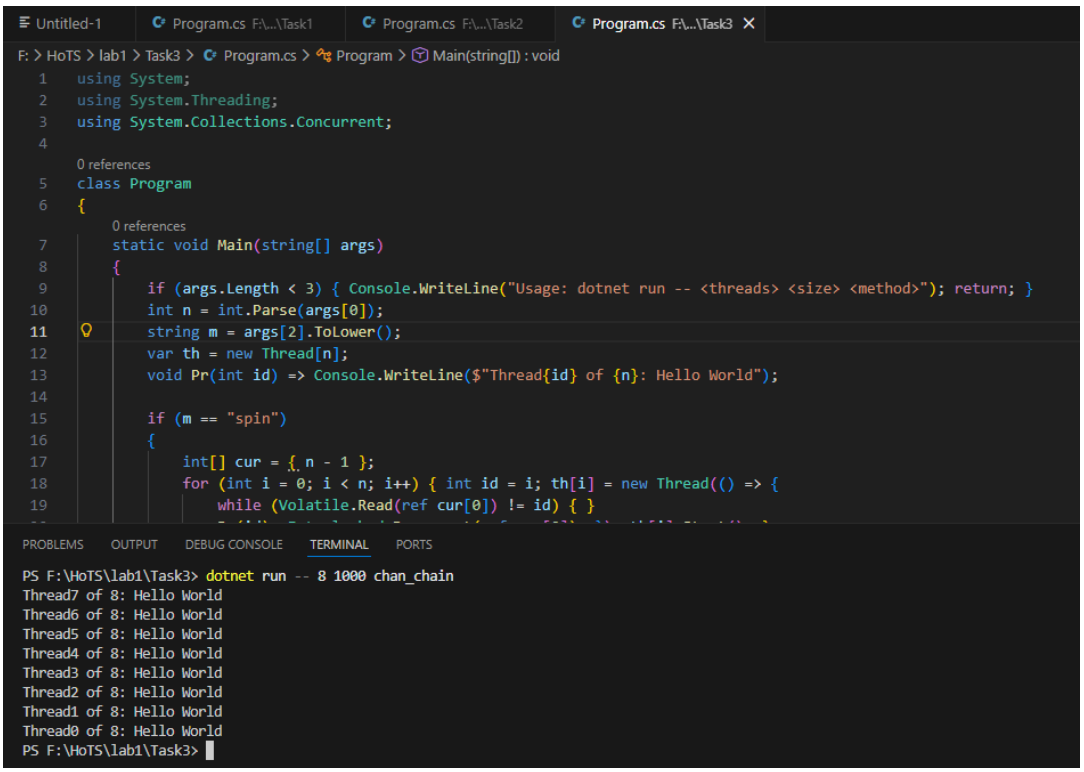



*Рисунок 11. Способ 3 (цепочка каналов)*

На рисунке 12 — способ 4 (координатор с массивом семафоров). Главный поток выдаёт разрешения в порядке 7, 6, …, 0 и дожидается подтверждения каждого потока — наиболее жёсткий контроль очерёдности.

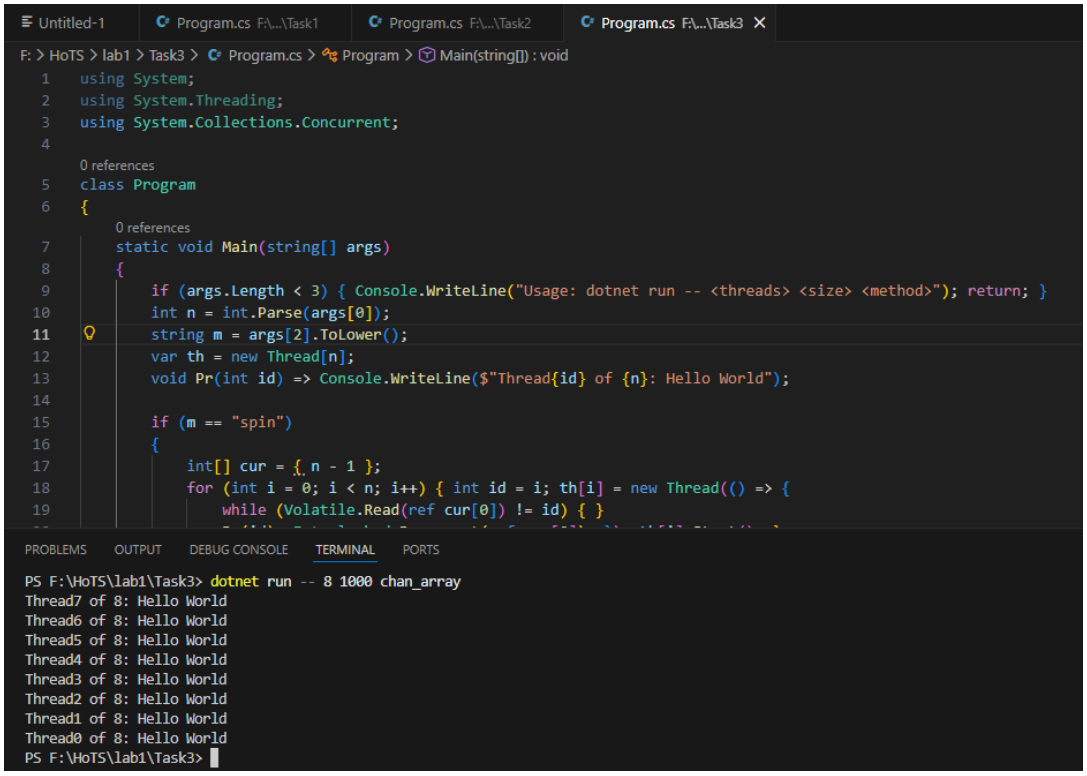



*Рисунок 12. Способ 4 (координатор с массивом семафоров)*

На рисунке 13 — способ 5 (цепочка ManualResetEventSlim). Каждый поток ждёт завершения потока с большим номером; порядок вывода строго обратный.

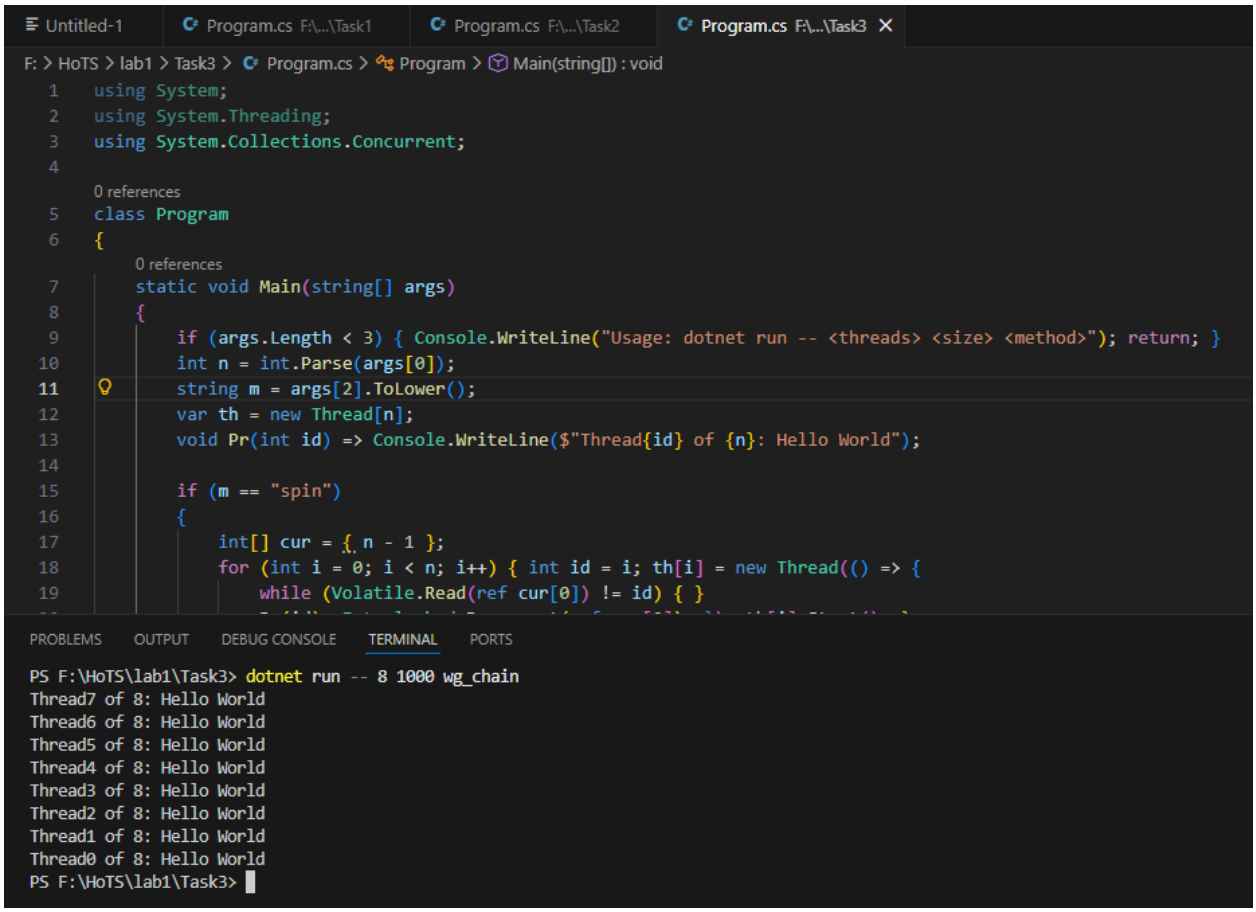



*Рисунок 13. Способ 5 (цепочка событий)*

Вопросы:

1 Задача. Всегда ли вывод идентичен? Почему?

Нет, не всегда. Потоки исполняются параллельно, и порядок достижения ими оператора вывода выбирается планировщиком ОС, а не программой. Во всех запусках печатается один и тот же набор строк (по одной от каждого из 8 потоков), но их последовательность меняется, что видно на рисунках 1–3. lock защищает лишь атомарность записи строки и не вносит упорядоченности между потоками.

3 Задача. Существует как минимум 5 способов решения.

Реализовано и продемонстрировано пять способов фиксации обратного порядка: (1) активное ожидание на атомарном счётчике; (2) условная переменная Monitor.Wait/PulseAll; (3) цепочка каналов с передачей эстафеты;

(4) координатор с массивом семафоров и рукопожатием; (5) цепочка событий ManualResetEventSlim. Все пять версий дали корректный порядок 7, 6, …, 0 (рисунки 9–13). Аналогом упорядоченного вывода в OpenMP служит директива ordered. Прочие приёмы (например, искусственные задержки, зависящие от номера потока) порядок не гарантируют и в работе не применялись.

Вывод: В задаче 1 построена параллельная область с идентификацией потоков на Thread и барьером через Join(). Установлено, что без синхронизации порядок вывода недетерминирован: набор строк постоянен во всех запусках, а их последовательность определяется планировщиком ОС.

В задаче 2 реализованы и сопоставлены типы распределения работ static (с блоком и без), dynamic, guided и runtime. Подтверждено, что при независимых итерациях результат не зависит от расписания (контрольная сумма совпала во всех запусках), а различаются балансировка загрузки и время выполнения.

В задаче 3 выполнено пять способов синхронизации, фиксирующих обратный порядок вывода, и сравнены их свойства: активное ожидание расходует процессорное время, условные переменные, каналы и события позволяют ждать без нагрузки на CPU, координатор даёт наиболее жёсткий контроль очерёдности.